

The AI Datacenter Network

From Enterprise Networking to Distributed GPU Fabrics

A comprehensive training guide for senior enterprise and service-provider network engineers preparing for a **Senior AI Datacenter Network Engineer** role.

Purpose: build the depth needed to speak intelligently with hiring managers and technical teams about RDMA fabrics, lossless Ethernet, GPU collective traffic, 400G/800G physical plant, host networking, and fabric operations.

July 2026 · Prepared with source material on AI fabric skill gaps

Contents

1	The Mental Shift: The Network Is Part of the Computer	4
1.1	Why this role is different	4
1.2	The economics that drive the engineering	4
1.3	What actually changes for you	4
1.4	Translating your enterprise experience	5
1.5	The role in one sentence	5
2	RDMA, InfiniBand, and RoCEv2: The Transport Layer of AI	6
2.1	Why TCP thinking does not apply	6
2.2	RDMA verbs and queue pairs	6
2.3	InfiniBand: the native RDMA fabric	7
2.4	RoCEv2: RDMA over converged Ethernet	7
2.5	GPUDirect RDMA: taking the CPU out of the picture	7
2.6	Minimum vocabulary check	8
3	Lossless Ethernet: PFC, ECN, DCQCN, and Buffer Engineering	9
3.1	PFC: Priority Flow Control (IEEE 802.1Qbb)	9
3.2	ECN and DCQCN: the actual congestion control	9
3.3	DSCP-to-traffic-class mapping	10
3.4	Buffer architecture and microbursts	10
3.5	Putting it together: the lossless design conversation	11
4	GPU Collective Traffic: What the Workload Does to Your Fabric	12
4.1	NCCL: the traffic generator	12
4.2	The collective operations	12
4.3	Parallelism strategies and their network signatures	13
4.4	Traffic characteristics that break enterprise assumptions	13
4.5	The failure-mode view	14
5	AI Fabric Topology: Beyond Generic Leaf-Spine	15
5.1	Foundations: Clos, bisection, and oversubscription	15
5.2	Rail-optimized design	15
5.3	Multi-plane fabrics and NVLink domain boundaries	15
5.4	Failure domains and placement	16
5.5	Load balancing: the ECMP problem and its fixes	16
6	The 400G/800G Physical Layer: Where Performance Problems Are Born	17
6.1	Form factors and cabling options	17
6.2	FEC and BER: reading link health like a pro	17
6.3	DOM telemetry, link training, and flap forensics	18
6.4	Breakouts, polarity, and the cabling discipline	18
7	The Host Is Part of the Network: NICs, Drivers, NUMA, and PCIe	19
7.1	The hardware and software stack	19
7.2	The toolbox — commands and what each tells you	19
7.3	NUMA locality and PCIe topology	20
7.4	SR-IOV and multi-tenancy	20
7.5	A host-side triage playbook	20
8	AI Fabric Observability and Failure Analysis	22
8.1	The counter set that actually matters	22
8.2	Telemetry architecture	22
8.3	Worked example: “Why is this job 17% slower?”	23
8.4	Failure analysis culture	23
9	Orchestration, NOS Diversity, and Fabric-as-Code	24
9.1	Kubernetes: how the fabric reaches the pod	24
9.2	Slurm: the HPC scheduler view	24

9.3	NOS diversity: SONiC, whitebox, and life beyond appliance CLI	24
9.4	Fabric-as-code: the operating model	24
10	Facilities, DCI, and Where the Standards Are Going	26
10.1	Power, cooling, and rack reality	26
10.2	DCI for AI: connecting clusters and regions	26
10.3	Standards direction: Ultra Ethernet and the road ahead	26
11	Interview Preparation: Questions, Model Answers, and Positioning	28
11.1	How to position an enterprise background	28
11.2	Technical questions and model answers	28
11.3	Questions to ask the interviewer	30
11.4	Red flags to avoid in your own answers	31
12	A Practical Learning Path, Labs, Glossary, and References	32
12.1	The five-phase learning path	32
12.2	Glossary	33
12.3	References	34
12.4	Closing	34

1 The Mental Shift: The Network Is Part of the Computer

1.1 Why this role is different

If you come from enterprise or service-provider networking, you already have most of the raw material for this job: routing at scale, VXLAN/EVPN fabrics, QoS, telemetry, and automation. What changes is not the protocols — it is the **definition of success**.

An enterprise network exists to provide reliable connectivity, segmentation, security, and predictable operations to thousands of loosely coupled applications. If a link drops a few packets, TCP retransmits, users never notice, and the SLA dashboard stays green.

An AI training fabric exists for exactly one purpose: **to make thousands of GPUs behave like one giant computer with minimal idle time**. The network is the backplane of a distributed supercomputer. Distributed training is a tightly synchronized bulk-synchronous workload: every GPU computes, then every GPU exchanges gradients or activations, then every GPU waits until that exchange completes before the next step begins. The job advances at the pace of the **slowest** participant.

KEY CONCEPT

In enterprise networking, packet loss is an inconvenience absorbed by TCP. In an AI fabric, a single congested queue, a marginal optic, or one mis-cabled rail creates a **straggler** — and because collectives are barriers, one straggler stalls every GPU in the job. A cluster of 16,000 GPUs at roughly 2–3 dollars per GPU-hour burns tens of thousands of dollars per hour of aggregate idle time. Tail latency is not a nicety here; it is the product.

1.2 The economics that drive the engineering

Every design decision in this domain traces back to one number: **GPU utilization** (often discussed as MFU — model FLOPS utilization — or “goodput” at the job level). The network is typically 10–15 percent of cluster capital cost, but a poorly behaving network can idle the other 85–90 percent. That asymmetry explains behaviors that look extreme by enterprise standards:

- Building **non-blocking or lightly oversubscribed** fabrics when enterprise designs happily run 4:1 or worse.
- Treating **microbursts, ECN mark rates, and PFC pause counters** as first-class operational signals, not curiosities.
- Replacing optics on **rising correctable FEC error trends**, long before the link actually flaps.
- Running **synthetic RDMA and NCCL benchmarks before and after every change**, the way an enterprise team might run a ping sweep.
- Refusing “it’s up, ping works” as a definition of healthy. The bar is: **the job is not network-stalled**.

1.3 What actually changes for you

Dimension	Enterprise mindset	AI fabric mindset
Success metric	Availability, five nines, MTTR	Job completion time, GPU idle time, step-time variance
Loss	TCP recovers; some loss is normal	Loss collapses RDMA throughput; fabric engineered lossless or loss-tolerant by design
Latency	Averages matter	P99.9 (tail) matters; collectives finish at the slowest flow
Flows	Millions of small flows; ECMP hashes well	Few, huge, synchronized elephant flows; ECMP hash collisions are a core problem
QoS	Voice/video priority queues	PFC + ECN engineered per traffic class; buffer thresholds tuned per platform

Dimension	Enterprise mindset	AI fabric mindset
Monitoring	Port up/down, utilization, syslog	Queue depth, pause frames, ECN marks, FEC histograms, NCCL timing correlation
Change control	Maintenance windows, manual CLI	GitOps pipelines, CI validation, automated pre/post performance baselines
The host	Someone else's problem	NIC firmware, drivers, NUMA, and PCIe topology are part of the network

1.4 Translating your enterprise experience

Hiring managers are not looking for someone who abandons enterprise discipline. They want that discipline **applied to a new failure model**. Here is how existing senior-level skills map:

You already know	How it translates to the AI datacenter
BGP / core routing	BGP is the standard underlay for large RoCE fabrics (RFC 7938-style eBGP per switch, unnumbered interfaces, /31s or IPv6 link-locals). Your routing depth transfers almost directly to the scale-out fabric underlay and to DCI.
VXLAN/EVPN	Still heavily used on the front-end network (management, storage access, user traffic, multi-tenancy). Back-end GPU fabrics are usually plain routed IP — simpler, but your overlay skills remain valuable for the rest of the plant.
QoS experience	Directly foundational for PFC, ECN, DSCP-to-traffic-class mapping, and buffer tuning — the single most important new skill area (Chapter 3).
Telemetry, Splunk/Zabbix	Evolves into gNMI streaming telemetry at seconds-level cadence, queue watermarks, and correlating fabric counters with GPU job metrics (Chapter 8).
Automation / IaC	Becomes mandatory. 1,000–100,000 GPU fabrics are operated as code: source-of-truth-driven config, CI validation, automated cabling checks (Chapter 9).
Optical / DWDM exposure	Maps to 400G/800G optics, FEC analysis, and coherent DCI (Chapters 6 and 10).

INTERVIEW SOUNDBITE

“The biggest shift coming from enterprise is redefining what ‘working’ means. In my world, loss was absorbed by TCP and averages mattered. In an AI fabric, the network is the backplane of one distributed computer — collectives are synchronization barriers, so tail latency and microbursts directly convert into GPU idle time and dollars. My job is not to keep ports up; it is to keep the job from being network-stalled.”

1.5 The role in one sentence

A senior AI network engineer is **part network engineer, part HPC fabric engineer, part Linux/NIC troubleshooter, part automation engineer, and part performance analyst**. The rest of this guide builds each of those parts in the order a fabric is built: transport semantics first, then losslessness, then the workload, then topology, then the physical plant, then the host, then operations.

2 RDMA, InfiniBand, and RoCEv2: The Transport Layer of AI

2.1 Why TCP thinking does not apply

Classic TCP/IP networking pays three taxes that AI workloads cannot afford: the kernel (system calls, context switches, interrupt handling), memory copies (user buffer to kernel socket buffer to NIC), and CPU cycles spent shuffling bytes instead of computing. At 400 or 800 Gb/s per GPU-attached NIC, a software network stack cannot keep up, and every CPU cycle spent on networking is stolen from data loading and preprocessing.

RDMA (Remote Direct Memory Access) eliminates all three taxes. The NIC hardware moves data directly between application memory on two servers — no kernel involvement in the data path, no intermediate copies, no per-packet CPU work. The application posts a work request; the NIC executes the transfer and signals completion.

KEY CONCEPT

RDMA is **hardware-offloaded, zero-copy, kernel-bypass** networking. The NIC is a DMA engine with a network port. Consequently, much of “the network stack” now lives in NIC firmware and switch buffer behavior — which is why AI network engineers must own the NIC and host stack, not just the switches.

2.2 RDMA verbs and queue pairs

RDMA is programmed through the **verbs** API (provided by `rdma-core` / `libibverbs`). The concepts appear constantly in troubleshooting and interviews:

- **Queue Pair (QP)** — the RDMA equivalent of a socket: a Send Queue and a Receive Queue in NIC-managed memory. Applications post **Work Requests** (WRs) to a QP; the NIC executes them asynchronously.
- **Completion Queue (CQ)** — where the NIC reports finished work. Polling the CQ replaces interrupts for latency-critical paths.
- **Memory Registration (MR)** — before the NIC may touch a buffer, the buffer is registered: pinned in physical memory and given local/remote keys (`lkey` / `rkey`). Registration is expensive, so applications register large regions once and reuse them.
- **Protection Domain (PD)** — a container that scopes which QPs may use which MRs.

Operations come in two flavors:

- **Two-sided (SEND/RECV)** — the receiver must pre-post a receive buffer; both sides’ CPUs are minimally involved. Semantically like message passing.
- **One-sided (RDMA WRITE / RDMA READ / ATOMIC)** — the initiator reads or writes the remote host’s registered memory directly. The remote CPU is **not involved at all**. This is what makes RDMA feel fundamentally different from sockets: memory access semantics over a network.

QP transport types matter operationally:

- **RC (Reliable Connection)** — connected, ordered, acknowledged, hardware retransmission. What NCCL and most training traffic uses today. Go-back-N retransmission on loss is why loss is so expensive (Chapter 3).
- **UC (Unreliable Connection)** and **UD (Unreliable Datagram)** — niche; UD underpins connection management and some HPC patterns.
- **DC (Dynamically Connected)** — an NVIDIA/Mellanox extension that reduces QP memory footprint at very large scale.

FIELD GOTCHA

RC transport traditionally uses **go-back-N**: a single lost packet forces retransmission of the entire window from that point. One drop can cost megabytes of retransmitted data and a latency spike measured in milliseconds — an eternity for a synchronized collective. This is the root reason AI Ethernet fabrics are engineered lossless

(PFC/ECN) and why newer stacks (ConnectX selective repeat, Ultra Ethernet Transport) move to selective retransmission.

2.3 InfiniBand: the native RDMA fabric

InfiniBand (IB) is a complete, purpose-built stack — physical layer through transport — where RDMA is native, and losslessness is a design property rather than a configuration exercise.

Key architectural facts a senior candidate should command:

- **Credit-based flow control.** An IB transmitter may only send when the receiver has advertised buffer credits, per **virtual lane (VL)**. Loss from congestion is architecturally impossible — no PFC tuning required. This is IB's single biggest operational advantage.
- **Subnet Manager (SM).** A centralized controller (OpenSM, or NVIDIA UFM in production) discovers the topology, assigns **LIDs** (local identifiers), computes routing tables for every switch, and monitors the fabric. Switches themselves are simple and fast. Contrast with Ethernet's fully distributed control plane — the SM is both a strength (global optimization) and an operational discipline (SM redundancy, planned failover).
- **Adaptive routing.** Because the SM sees the whole fabric, IB switches can spray traffic across multiple equal paths and route around congestion per-packet, with the destination NIC handling reordering.
- **SHARP (Scalable Hierarchical Aggregation and Reduction Protocol).** In-network computing: the switches themselves perform the reduction arithmetic of all-reduce, so gradient data is combined **in the fabric** rather than ping-ponged between GPUs. Cuts collective traffic volume roughly in half and reduces latency at scale.
- **Current generation.** NVIDIA Quantum-X800 switching provides 800 Gb/s ports with adaptive routing, telemetry-based congestion control, and SHARP v4 — paired with ConnectX-8 adapters.

2.4 RoCEv2: RDMA over converged Ethernet

RoCEv2 takes the InfiniBand transport layer (the same IB Base Transport Header, the same verbs semantics) and encapsulates it in **UDP/IP over Ethernet**, destination UDP port 4791. Because it is routable IP, it runs over a standard L3 Clos fabric — your BGP skills apply directly to the underlay.

What RoCEv2 inherits and what it must borrow:

- It inherits IB's transport semantics — including RC's intolerance of loss.
- It does **not** inherit IB's credit-based link layer. Ethernet drops on congestion by default. Therefore the Ethernet fabric must be **made** lossless (or nearly so) using PFC and ECN/DCQCN — the subject of Chapter 3, and the heart of this job.
- Entropy for load balancing comes from the **UDP source port**, which the NIC varies per QP/flow — important later when we discuss ECMP.

The industry positioning to be fluent in: NVIDIA's Spectrum-X Ethernet platform is RoCE between GPU servers with NIC/switch co-engineered adaptive routing and congestion control; Arista/Broadcom publish AI deployment guides framing RoCE as hardware-offloaded RDMA over Ethernet on standard merchant silicon; and the Ultra Ethernet Consortium is standardizing the next-generation transport (Chapter 10). Ethernet won the volume argument; IB still holds the turnkey-performance argument. A senior engineer can argue both sides (see the interview chapter).

2.5 GPUDirect RDMA: taking the CPU out of the picture

Without GPUDirect, moving GPU data between servers means: GPU memory to host RAM (bounce buffer), host RAM to NIC, then the reverse on the far side — burning PCIe bandwidth, CPU, and latency twice.

GPUDirect RDMA lets the NIC DMA directly into and out of **GPU HBM** across PCIe, with the CPU and host memory completely out of the data path. The NIC and GPU exchange data peer-to-peer, ideally under the same PCIe switch. Requirements you will be expected to know:

- NVIDIA GPU + RDMA-capable NIC (ConnectX-class) with compatible driver stack (`nvidia-peermem` or DMA-BUF on modern kernels).

- Sane **PCIe topology**: NIC and GPU on the same root complex or, ideally, same PCIe switch. `nvidia-smi topo -m` shows the relationship (PIX/PXB good; PHB/SYS bad).
- **ACS (Access Control Services)** on intermediate PCIe switches must permit peer-to-peer, or traffic silently detours through the root complex and performance collapses (a classic field issue — Chapter 7).

Related family members worth name-dropping accurately: **GPUDirect Storage** (NVMe/storage DMA direct to GPU) and **GDRCopy** (low-latency CPU-mapped GPU memory access).

INTERVIEW SOUNDBITE

“RoCEv2 is literally the InfiniBand transport in a UDP/IP envelope — same verbs, same queue pairs, same reliable-connection semantics, and the same intolerance of packet loss. The difference is that InfiniBand gets losslessness from credit-based flow control by design, while with RoCE I have to engineer losslessness on Ethernet with PFC and ECN, and then monitor that it stays engineered. That is the core of the AI network engineer’s job on an Ethernet fabric.”

2.6 Minimum vocabulary check

Before moving on, you should be able to define from memory: verbs, QP, WQE, CQ, MR, `lkey` / `rkey`, RC vs UD, one-sided vs two-sided, go-back-N, VL, LID, SM, SHARP, BTH, UDP 4791, GPUDirect RDMA, and peer-to-peer PCIe. These terms **will** come up in a technical screen.

3 Lossless Ethernet: PFC, ECN, DCQCN, and Buffer Engineering

This is the single most important new skill area for an engineer coming from enterprise switching. Everything here exists to solve one problem: **RoCE transport punishes loss brutally, and Ethernet drops by default**. The solution is a two-layer system — ECN-based congestion control as the primary mechanism, PFC as the last-resort safety net — plus careful buffer and QoS engineering to make both behave.

3.1 PFC: Priority Flow Control (IEEE 802.1Qbb)

PFC is per-priority pause. When a switch ingress queue for a given priority crosses its **XOFF threshold**, the switch sends a PFC pause frame upstream naming that priority; the upstream device stops transmitting **that priority only** until an XON resumes it. Done correctly, frames wait in upstream buffers instead of being dropped.

The mechanics you must be able to whiteboard:

- PFC operates **per traffic class** on a link — pausing the RoCE class must not touch management, storage, or CNP traffic. This is why DSCP-to-TC mapping (below) must be consistent fabric-wide.
- **Headroom buffer**. After sending XOFF, in-flight bytes keep arriving for one round trip (cable propagation both ways + peer response time + MTU-sized frames already serializing). The ingress must reserve **headroom** per port per priority to absorb them. Headroom scales with link speed and **cable length** — a 100 m AOC needs materially more headroom than a 2 m DAC. Getting headroom wrong means drops **despite** PFC being “on.”
- Thresholds are typically dynamic (a fraction “alpha” of remaining shared buffer) rather than static, so a busy switch tightens pause behavior as the shared pool depletes.

3.1.1 Why PFC is a safety net, not a control system

PFC is **blunt**: it stops everything in the class at the previous hop, punishing every flow that shares the priority, not just the one causing congestion.

- **Head-of-line blocking**. One congested egress can pause an ingress that also feeds uncongested egresses — innocent flows stall.
- **Pause propagation / pause storms**. The paused upstream switch’s buffers fill, so **it** pauses **its** upstreams. Congestion propagates backward hop by hop, and a single misbehaving endpoint (a slow NIC, a broken receiver that stops draining) can cascade pauses across a large region of the fabric. A malfunctioning NIC that emits continuous pause frames can measurably degrade an entire pod.
- **PFC deadlock**. If buffer dependencies form a cycle (possible with certain routing/failure combinations in Clos fabrics), every switch in the cycle waits for the next to release credit — permanent standstill until something breaks the cycle. Mitigations: keep RoCE traffic on strictly up-down (valley-free) paths, and enable the **PFC watchdog** — which detects a queue paused beyond a threshold (typically hundreds of milliseconds) and force-drains or ignores pause to break the deadlock, trading a burst of loss for fabric liveness.

FIELD GOTCHA

The classic operational trap: PFC counters look “normal” because pauses are being sent and honored — but job performance is terrible. Pause frames are not a sign of health; they are a sign ECN failed to keep queues short. A well-tuned fabric shows **ECN marks routinely, PFC pauses rarely**. Rising sustained PFC pause counters are an alarm, not background noise.

3.2 ECN and DCQCN: the actual congestion control

ECN (Explicit Congestion Notification) uses two bits in the IP header. Fabric switches are configured with WRED/ECN thresholds per queue: as queue depth crosses the minimum threshold, the switch begins **marking** packets (CE — Congestion Experienced) with rising probability instead of dropping them.

DCQCN (Data Center Quantized Congestion Notification) is the end-to-end control loop built on those marks — the de facto congestion control for RoCEv2, implemented in NIC hardware:

1. **CP (Congestion Point)** — the switch marks CE on packets as its queue grows.
2. **NP (Notification Point)** — the **receiving NIC** sees CE-marked RoCE packets and sends a **CNP (Congestion Notification Packet)** back to the sender for that flow.
3. **RP (Reaction Point)** — the **sending NIC** receives the CNP and cuts that QP's transmit rate sharply, then recovers with additive increase plus fast-recovery stages, governed by an internal "alpha" congestion estimate that decays when CNPs stop arriving.

Design and tuning points that distinguish a senior candidate:

- **CNPs must travel in a protected high-priority class.** If congestion feedback itself gets stuck behind the congestion it reports, the loop fails. Standard practice: RoCE data in one class (commonly DSCP 26, queue 3), CNPs in a strict-priority class (commonly DSCP 48, queue 6).
- **ECN thresholds (Kmin/Kmax/Pmax) tune the trade-off.** Mark too early: NICs throttle prematurely and you leave bandwidth idle. Mark too late: queues grow, latency rises, and PFC starts firing. Vendor AI deployment guides (Arista/Broadcom, NVIDIA, Cisco) publish per-ASIC starting values; fleets then tune from telemetry.
- **ECN before PFC — always.** The whole design intent is that ECN thresholds sit **below** PFC XOFF thresholds so the rate-control loop resolves congestion before pause is ever needed. If you observe pauses before meaningful mark rates, your thresholds are inverted or misconfigured.
- Timescale intuition: DCQCN reacts at network RTT timescales (microseconds); PFC reacts at link RTT (sub-microsecond); buffers absorb what both miss.

3.3 DSCP-to-traffic-class mapping

Lossless behavior only works if classification is consistent on **every** port of **every** switch and NIC in the fabric. A representative RoCE QoS plan:

Traffic	DSCP	Queue/TC	Treatment
RoCE data (GPU-to-GPU)	26	3	Lossless: PFC enabled, ECN marking, guaranteed bandwidth (e.g., WRR 50%+)
CNP (congestion feedback)	48	6	Strict priority, no PFC, never delayed
Management / underlay control (BGP, gNMI)	48/56	7	Strict or protected queue
Storage / checkpoint traffic	varies	dedicated	Often lossy-but-weighted, or its own lossless class — design decision
Everything else	0	0	Best effort

FIELD GOTCHA

A single switch port with a missing or mismatched DSCP-to-TC map silently declassifies RoCE into the best-effort queue on that hop — no PFC, no ECN. Result: intermittent drops under load, RDMA retransmits, and a "17% slower job" with every port nominally up. Fabric-wide QoS config validation belongs in your CI pipeline (Chapter 9), not in a human's memory.

3.4 Buffer architecture and microbursts

Understanding your ASIC's buffer model is what separates configuring QoS from **engineering** it:

- Datacenter switch ASICs (e.g., Broadcom Tomahawk class, NVIDIA Spectrum) have tens of MB of **shared** on-chip buffer, partitioned into reserved per-port/per-class minimums, a shared dynamic pool governed by alpha values, and PFC headroom. Deep-buffer ASICs (Broadcom Jericho class, multi-GB external HBM/packet memory) appear mainly at DCI and storage boundaries, not usually inside the GPU fabric.
- **Microbursts** are the defining stress: collectives make hundreds of NICs transmit line-rate bursts **simultaneously** (Chapter 4), and multiple 400G/800G flows can converge on one egress for microseconds. Average utilization

graphs at 30-second polling show 40% while queues overflow at microsecond timescales. This is why watermark counters, high-frequency telemetry, and burst-aware monitoring matter (Chapter 8).

- **Incast** is the pathological pattern: N senders, one receiver (common in reduce and checkpoint phases). Buffer occupancy spikes at N times the per-sender burst; ECN/DCQCN plus sane collective algorithms keep it survivable.

3.5 Putting it together: the lossless design conversation

A hiring manager asking “how would you design the RoCE fabric” wants to hear a layered answer:

1. Consistent DSCP/TC classification and trust settings fabric-wide, from NIC to every switch hop.
2. ECN/WRED thresholds set per vendor AI guidance for the ASIC, positioned to act **before** PFC.
3. PFC enabled on the RoCE class only, with correctly sized per-cable-length headroom, watchdog enabled, and pause counters alarmed.
4. CNPs in a strict-priority class.
5. Adaptive/dynamic load balancing or packet spraying to prevent ECMP hot links (Chapter 5).
6. Continuous telemetry on marks, pauses, watermarks, and drops, validated by synthetic RDMA tests in CI.

INTERVIEW SOUNDBITE

“I think of lossless Ethernet as a control system with a safety valve. DCQCN — switch ECN marking driving NIC rate control via CNPs — is the control system that keeps queues short. PFC is the safety valve for the microbursts the control loop can’t catch in time. If I see PFC firing regularly, I don’t celebrate that it prevented drops — I go tune why ECN didn’t keep us out of the pause regime. And I always keep the failure modes in mind: pause storms, head-of-line blocking, and PFC deadlock, which is why the watchdog and valley-free routing are non-negotiable.”

4 GPU Collective Traffic: What the Workload Does to Your Fabric

You cannot design or troubleshoot an AI fabric without knowing what distributed training actually transmits. This chapter is the “know your customer” chapter: NCCL, the collective operations, the parallelism strategies that generate them, and the traffic signatures they leave on the wire.

4.1 NCCL: the traffic generator

NCCL (NVIDIA Collective Communications Library) is the communication engine underneath PyTorch and JAX distributed training on NVIDIA hardware. It implements collective operations across GPUs within a server (over NVLink) and across servers (over IB or RoCE), and it is **topology-aware**: at startup it discovers NVLink connectivity, PCIe layout, NIC placement, and rails, then builds ring and tree communication graphs over that topology.

Two consequences make NCCL your concern as a network engineer:

1. **The network topology directly shapes job performance**, because NCCL’s rings and trees are laid over the physical paths you built. A rail-optimized fabric (Chapter 5) exists specifically to serve NCCL’s communication patterns.
2. **NCCL behavior is tunable and mis-tunable** via environment variables (`NCCL_ALGO` , `NCCL_PROTO` , `NCCL_IB_HCA` , `NCCL_SOCKET_IFNAME` , `NCCL_IB_QPS_PER_CONNECTION` , `NCCL_MIN_CTAS` ...). “The network is slow” tickets are frequently NCCL picking a wrong interface or algorithm — you must be able to read NCCL debug output (`NCCL_DEBUG=INFO`) and the `nccl-tests` benchmarks.

4.2 The collective operations

For a data size S per GPU across N GPUs, know what each collective does and what it costs:

Collective	What it does	Where you see it
All-reduce	Every GPU ends with the element-wise sum of all GPUs’ buffers. Equivalent to reduce-scatter + all-gather. Each GPU sends/receives about $2S \times (N-1)/N$.	Gradient synchronization in data parallelism — the classic training collective
Reduce-scatter	Sum is computed but each GPU keeps only its $1/N$ shard.	First half of all-reduce; ZeRO/FSDP gradient sharding
All-gather	Every GPU’s shard is concatenated to every GPU.	FSDP/ZeRO parameter gathering before each layer’s compute
Broadcast	One GPU’s buffer copied to all.	Weight initialization, checkpoint restore fan-out
All-to-all	Every GPU sends a distinct block to every other GPU — $N \times (N-1)$ simultaneous flows.	Mixture-of-Experts routing; the most fabric-hostile pattern
Point-to-point send/recv	Pairwise transfers.	Pipeline parallelism activations between stages

4.2.1 Ring vs tree all-reduce

- **Ring**: GPUs form a logical ring; data moves in chunks around it in $2(N-1)$ steps. Bandwidth-optimal (each link carries the minimum possible), so it dominates for large messages. Cost: latency grows linearly with N — at thousands of GPUs, ring latency for small messages hurts.
- **Tree**: logarithmic depth, latency-optimal for small/medium messages at large scale. NCCL switches between rings, trees, and hybrid algorithms per message size and scale.
- **SHARP/in-network reduction** (IB, and switch-assisted collectives elsewhere) offloads the arithmetic into switches, roughly halving traffic and cutting latency — worth raising in any IB-vs-Ethernet discussion.

The `nccl-tests` suite (`all_reduce_perf` etc.) reports **algorithm bandwidth** (data size / time) and **bus bandwidth** (normalized to per-link utilization so it is comparable to line rate). Bus bandwidth at 85–95% of line rate is healthy; interpreting these numbers is a practical skill hiring teams probe.

4.3 Parallelism strategies and their network signatures

Large-model training composes several parallelism dimensions, and each stresses the network differently. This mapping is core senior-level knowledge:

Strategy	What it splits	Network signature
Data parallel (DP)	Same model replicated; each GPU gets different data; gradients synchronized every step.	Periodic large all-reduce across the DP group — big, bursty, synchronized; tolerant of moderate latency, hungry for bandwidth.
Fully sharded DP (FSDP/ZeRO)	Parameters, gradients, optimizer state sharded across GPUs.	All-gather + reduce-scatter per layer , overlapped with compute — more frequent, finer-grained collectives; sensitive to latency and jitter.
Tensor parallel (TP)	Individual layers' matrices split across GPUs.	All-reduce/all-gather inside every layer, multiple times per step — extremely latency-sensitive; kept inside the NVLink domain (within a server or NVL72 rack), almost never across your fabric.
Pipeline parallel (PP)	Model layers split into sequential stages.	Point-to-point activation/gradient transfers between adjacent stages — moderate bandwidth, steady cadence.
Expert parallel (MoE)	Different experts on different GPUs; tokens routed dynamically.	All-to-all at every MoE layer — dense any-to-any mesh of flows, unpredictable per-destination volume; the pattern that most punishes oversubscription and poor load balancing.

KEY CONCEPT

The placement rule that shapes cluster design: **the most latency-sensitive parallelism (TP) stays inside the NVLink domain; bandwidth-heavy but latency-tolerant parallelism (DP/PP) crosses the scale-out fabric**. Your fabric is primarily carrying DP all-reduce, FSDP all-gather/reduce-scatter, PP point-to-point, and MoE all-to-all. Schedulers and NCCL assume this hierarchy — and a job placed across failure domains that violates it will underperform on a perfectly healthy network.

4.4 Traffic characteristics that break enterprise assumptions

- **Synchronized bursts, not statistical multiplexing.** Enterprise fabrics work because millions of independent flows average out. Training traffic is the opposite: a compute phase of near-silence, then **every NIC transmits at line rate simultaneously** when the collective starts. Design for coordinated microbursts, not for averages.
- **Few, fat, long-lived flows with low entropy.** A GPU server pair may exchange a handful of enormous QP flows. Five-tuple ECMP hashing was designed for flow-count diversity that simply is not there — hash collisions park two elephants on one uplink while its neighbor idles (Chapter 5).
- **Barrier semantics.** Every step ends with synchronization. The slowest flow sets the pace for all N GPUs — this is why P99.9 flow-completion time, not average throughput, is the design metric.
- **Periodicity.** Steps repeat every few hundred milliseconds to seconds, so fabric pathologies show up as **rhythmic** congestion with the job's step frequency — a genuinely useful diagnostic signature.
- **Checkpoint traffic.** Every checkpoint interval, the job dumps model + optimizer state (from hundreds of GB to tens of TB for frontier models) to storage — a massive synchronized **write** burst on the storage network. If

checkpoints and gradients share links or buffers, checkpoints can stall training; many designs isolate a storage/front-end fabric from the GPU back-end fabric partly for this reason.

4.5 The failure-mode view

Because of barrier semantics, workload symptoms map to network causes in characteristic ways:

- One degraded link (FEC storm, flapping optic) → one slow rail → stragglers → **every** step slower.
- ECMP collision → persistent hot uplink → specific ranks always late → step-time variance.
- A single “slow NIC” (thermal throttling, wrong PCIe width) → that rank lags every collective → job-wide slowdown that looks like “the network.”
- Lost RDMA packets (QoS misconfig on one hop) → go-back-N retransmit stalls → sawtooth throughput and `out_of_sequence` counters climbing on NICs.

INTERVIEW SOUNDBITE

“Training traffic inverts every statistical assumption enterprise QoS relies on: instead of millions of uncorrelated small flows, I get a few thousand synchronized elephant flows that all start in the same microsecond, because NCCL collectives are barriers. So I design for the burst, not the average — and when a job ‘gets slow’, my first mental model is a straggler: one rail, one link, one NIC dragging every step, not the fabric being uniformly busy.”

5 AI Fabric Topology: Beyond Generic Leaf-Spine

Enterprise leaf-spine answers the question “how does every server get network access?” AI fabric design answers a different question: “**how do I preserve GPU-to-GPU bandwidth for a specific, known communication pattern at minimum cost and blast radius?**”

5.1 Foundations: Clos, bisection, and oversubscription

The scale-out fabric is a **fat-tree** / **folded Clos**: leaves (ToR or in-rack), spines, and at large scale a third **core/super-spine** tier connecting pods. Key quantitative vocabulary:

- **Bisection bandwidth** — cut the cluster into two halves; the bandwidth crossing the cut. A **non-blocking (1:1)** fabric provides full bisection: any half can talk to the other half at full NIC rate.
- **Oversubscription** — the ratio of downlink to uplink capacity at a tier. A leaf with 32 x 800G down and 16 x 800G up is 2:1.
- The engineering rule of thumb: **the GPU back-end fabric is built 1:1 through at least the pod level**, because all-reduce and all-to-all genuinely use the bisection. Front-end/storage networks can tolerate oversubscription; the gradient fabric mostly cannot. Where budgets force oversubscription, it is placed **between** pods, and the scheduler keeps jobs inside pods.

Worked example you can do on a whiteboard: 1,024 GPUs with one 400G NIC each = 409.6 Tb/s of NIC capacity. Non-blocking two-tier fabric with 64-port 800G switches (running ports as 2 x 400G toward hosts): each leaf uses 32 ports down / 32 up, needing 32 leaves and 16 spines for a 512-GPU pod at 1:1; two such pods join at a core tier sized to the inter-pod traffic you intend to allow. The point of the exercise: **fabric sizing is arithmetic from GPU count and NIC speed** — hiring panels love this question.

5.2 Rail-optimized design

The signature AI topology. In an 8-GPU HGX server, each GPU has its own dedicated 400G/800G NIC. Number the NICs 0–7. A **rail** is the set of NIC k across all servers.

- **Rail-optimized fabric**: NIC k of every server connects to leaf/plane k. Eight parallel “rails,” each its own subtree (or its own entire plane).
- **Why**: NCCL organizes inter-node communication so that GPU k on one server predominantly talks to GPU k on other servers (same rail). Same-rail traffic crosses **one leaf hop** instead of traversing spines — lower latency, fewer buffer contention points, and spine capacity reserved for the collectives that truly need it. Cross-rail traffic inside a server rides NVLink (NCCL’s PXN feature moves data to the right GPU over NVLink so it can exit on the right rail).
- **Operational consequence**: cabling and NIC-to-leaf mapping are **load-bearing correctness constraints**. One server with NIC 3 cabled into rail 5 forces cross-rail traffic through spines, and NCCL’s topology assumptions quietly degrade — “one mis-cabled rail can reduce millions of dollars of GPU productivity.” Automated cabling validation (LLDP against the design’s source of truth) is mandatory, not optional (Chapter 9).

5.3 Multi-plane fabrics and NVLink domain boundaries

- **Multi-plane**: instead of one giant fabric, build k independent parallel fabrics (“planes”), each connecting one NIC per server. Planes bound failure blast radius (a spine failure degrades one plane by 1/k, not the whole cluster), simplify very large scale-out, and map naturally onto rails.
- **NVLink domain**: inside a server (8 GPUs, 900 GB/s per GPU on NVLink4/NVSwitch) or a rack-scale NVL72 system (72 GPUs on an NVLink switch fabric), GPU-to-GPU bandwidth is roughly an order of magnitude above the network fabric. **The fabric’s job begins where the NVLink domain ends**. Cluster design is therefore hierarchical: NVLink domain → rail/leaf → pod (often a “scalable unit” of a few hundred to 1k GPUs at 1:1) → core tier between pods.
- **Alternative topologies**: dragonfly and dragonfly+ (HPC heritage, groups with all-to-all local links and global links between groups) trade cost for path diversity and appear in some IB and research designs; torus/mesh appears in TPU pods. For Ethernet AI fabrics, fat-tree/Clos with rails remains the dominant answer — but know the names and trade-offs.

5.4 Failure domains and placement

Because collectives are barriers, **where** a job lands matters as much as fabric health:

- Keep a job's DP/TP groups within the smallest enclosing domain (NVL domain → rail group → pod). A job spanning pods pays inter-pod oversubscription on every all-reduce.
- Schedulers (Slurm topology plugin, Kubernetes topology-aware scheduling) consume a topology description **you** help maintain — fabric engineering and job scheduling co-design (Chapter 9).
- Blast-radius thinking: a leaf failure in a rail design takes out one NIC on every server beneath it (jobs continue degraded on 7 of 8 rails) versus a whole server group on a conventional ToR. Design reviews in this world explicitly enumerate “what does the job feel” for each component failure.

5.5 Load balancing: the ECMP problem and its fixes

Static 5-tuple ECMP fails AI traffic (few flows, low entropy, elephant sizes — Chapter 4). Know the escalation ladder of fixes:

1. **More entropy from the NICs:** RoCE UDP source-port randomization per QP; NCCL splitting traffic across multiple QPs (`NCCL_IB_QPS_PER_CONNECTION`) to give the hash more material.
2. **Dynamic / adaptive load balancing (DLB):** switch reassigns flows (or **flowlets** — bursts separated by idle gaps) to the least-loaded uplink based on real-time egress load. Flowlet switching exploits collectives' natural pauses to re-path without reordering.
3. **Packet spraying / per-packet multipathing:** distribute every packet round-robin or load-aware across all uplinks — perfect utilization, but packets arrive out of order, so the **receiver must reorder**: modern NICs with DDP handle out-of-order RDMA placement (writing data directly where it belongs regardless of arrival order); Ultra Ethernet builds this in (Chapter 10). NVIDIA Spectrum-X markets exactly this combination — switch-side fine-grained spraying plus NIC-side reordering, with telemetry-driven congestion feedback.
4. **IB adaptive routing:** the InfiniBand equivalent, mature and SM-orchestrated, with destination-side reordering.

FIELD GOTCHA

A recurring real-world incident: fabric at 40% average utilization, one uplink at 100% with ECN marks and pauses — two elephant QPs hashed together. Enterprise instinct says “plenty of headroom”; AI fabric reality says “hash collision starving one rail.” The fix is adaptive load balancing or more QP entropy — not more bandwidth. Recognizing this pattern from counters is a standard interview scenario.

INTERVIEW SOUNDBITE

“I would describe a modern AI back-end as: NVLink domains at the bottom, rail-optimized leaves above them, non-blocking to the pod boundary, planes for blast-radius isolation, and only then oversubscription between pods — with the scheduler topology-aware so jobs stay inside their domain. And because the flow count is tiny by enterprise standards, I assume static ECMP will fail and plan for adaptive load balancing or packet spraying with NIC-side reordering from day one.”

6 The 400G/800G Physical Layer: Where Performance Problems Are Born

At 800G, the physical plant stops being a link-up/link-down concern and becomes a **performance** concern. Marginal links do not go down — they correct errors, add latency, drop the occasional frame, and quietly turn into stragglers. A senior AI network engineer is expected to be genuinely good at layer 1.

6.1 Form factors and cabling options

- **OSFP** — the dominant 800G form factor in AI fabrics (NVIDIA ecosystem especially). Larger, better thermals (often integrated heatsink fins, or “riding heatsink” variants); OSFP-RHS fits NIC cages.
- **QSFP-DD / QSFP-DD800** — backward-compatible with QSFP heritage; dominant in many Ethernet switch lines. You will encounter both ecosystems; port-cage-to-module compatibility is a real procurement and sparing issue.
- 800G today is **8 lanes of 100G PAM4** (or 4 x 200G emerging). PAM4 signaling (4 amplitude levels, 2 bits per symbol) is inherently noisier than NRZ — which is why FEC is mandatory (below).

Interconnect ladder, by reach and cost:

Type	What it is	Where it fits
DAC	Passive copper twinax	Cheapest, zero power, 1–2.5 m at 800G — in-rack only, thick and stiff (bend radius, airflow)
ACC/AEC	Active copper (redriven/retimed)	Extends copper to 3–7 m; AECs popular for in-row GPU-server-to-leaf
AOC	Active optical cable, fixed ends	Tens of meters, lighter than copper; failure means replacing the whole assembly
Transceivers + fiber	Pluggable optics over structured fiber (DR8/2xDR4 on parallel SMF via MPO, FR4/2xFR4 on duplex, VR/SR multimode short reach)	Row-to-row and fabric tiers; MPO-12/ MPO-16 APC connectors, polarity management, insertion-loss budgets
LPO	Linear pluggable optics — no DSP in module, host SerDes does equalization	Emerging: big power/latency savings per port, tighter host-module interop requirements
CPO	Co-packaged optics — optical engines beside the switch ASIC	The roadmap answer to optics power at 1.6T+; operational model still maturing

FIELD GOTCHA

Optics dominate fabric power and a surprising share of failures. An 800G module burns roughly 13–18 W; 64 of them is over a kilowatt **per switch** just for optics — with direct thermal consequences: optics running hot see rising bit-error rates and shortened life. Fleet experience consistently ranks optics/cabling among the top causes of link and job incidents. Treat optics as a managed fleet: inventory by part/firmware, DOM telemetry trended, thermal alarming, spares on site.

6.2 FEC and BER: reading link health like a pro

At PAM4 rates the raw channel is **designed** to have errors: pre-FEC BER around $1e-5$ to $1e-6$ is normal and **RS-544 FEC (Reed-Solomon “KP4”)** corrects it to a post-FEC target better than $1e-15$. This changes how you assess link health:

- **Pre-FEC (raw) BER / FEC corrected codewords** — your early-warning signal. A link whose corrected-error rate trends upward (temperature, aging laser, contaminated connector) is degrading **long before** anything drops.

- **Uncorrectable codewords** — actual frame loss. On an RDMA fabric each one can trigger go-back-N retransmission; even rates that round to zero percent loss create measurable job impact. Alarm on any sustained non-zero rate.
- **FEC symbol-error distribution / histograms** — bursts concentrated on one lane point to a specific fiber strand, connector, or SerDes lane rather than general degradation.
- **Per-lane view** — 800G is 8 lanes; a single bad lane (dirty MPO fiber position, marginal laser) shows as errors concentrated on one lane. Tools: `mlxlink` on NICs, vendor `show interfaces ... phy / transceiver detail`, `ethtool --show-fec` / `ethtool -S`.

FEC also buys its correction with **latency** (100 ns class) and with a cliff-edge failure mode: links look perfect until error rates exceed correction capacity, then degrade sharply.

6.3 DOM telemetry, link training, and flap forensics

- **DOM/DDM (digital optical monitoring)**: every module reports temperature, supply voltage, TX bias, TX power, and RX power per lane. Baseline at turn-up; trend continuously; alarm on drift (falling RX power = dirty/kinked fiber or dying far-end laser; rising temperature = airflow or seating problem).
- **Link training and autoneg**: at 400/800G, bring-up is a multi-second negotiation of equalizer settings per lane between SerDes, module DSPs, and the far end. Links that train slowly, or only after a specific power-cycle order, or negotiate down, indicate marginal channels **now** and flaps **later**. Firmware compatibility between NIC, module, and switch matters — matrix-test and pin versions.
- **Flap forensics**: a link flap on an AI fabric is a job-level event (NCCL communicator errors, job restart from checkpoint). For each flap collect: which end initiated (local fault vs remote fault), FEC/BER history before the event, DOM history, and correlation across links sharing a conduit, panel, or switch — one flapping optic is a component; twelve flapping ports on one panel is a physical-plant or power event.

6.4 Breakouts, polarity, and the cabling discipline

- **Breakout rules**: an 800G switch port often runs as 2 x 400G or 4 x 200G/8 x 100G toward hosts. ASICs group ports; using one breakout member can constrain its siblings' speeds, and breakout choice interacts with FEC mode and autoneg. Know your platform's port-group rules before design, not after cabling.
- **MPO/MTP hygiene**: multi-fiber push-on connectors are the workhorse — and the top contamination point. Polarity schemes (A/B/C), APC vs UPC, insertion-loss budgets across patch panels, and **inspect-and-clean before every mate** are non-negotiable disciplines. A single dusty MPO position = one bad lane = FEC bursts = straggler rail.
- **Rail cabling maps**: in a rail-optimized fabric the cable plan encodes the logical design (Chapter 5). The cabling documentation, LLDP-verified against the source of truth, is a first-class deliverable of fabric build — and re-verified after every physical intervention.

INTERVIEW SOUNDBITE

“At 800G I treat FEC counters the way I used to treat interface CRC errors — except the interesting signal is **pre-failure**: corrected-codeword trends, per-lane error skew, and DOM drift tell me which optic will hurt a job next week. On an RDMA fabric a link doesn't have to go down to cost money; a marginal optic that's 'up' is a straggler generator. So my physical-layer posture is: baseline everything at turn-up, trend DOM and FEC continuously, and replace on trend, not on failure.”

7 The Host Is Part of the Network: NICs, Drivers, NUMA, and PCIe

In enterprise networking the demarcation is the switchport. In AI infrastructure the network engineer's domain extends through the NIC firmware, the Linux RDMA stack, the PCIe topology, and up to the GPU's doorstep — because a fabric-perfect packet that dies on a mis-tuned host still stalls the job, and the ticket still says “network.”

7.1 The hardware and software stack

A GPU server (e.g., HGX class) pairs each GPU with a dedicated RDMA NIC — NVIDIA ConnectX-7/ConnectX-8 (or BlueField DPUs, which add Arm cores and offloads on top of the same ConnectX silicon). The Linux-side stack, top to bottom:

- Application / framework → NCCL → **verbs** (**libibverbs** , from **rdma-core**)
- Provider driver (**mlx5** kernel modules), NIC firmware
- **DOCA-OFED** / **MLNX_OFED** (NVIDIA's packaged stack) versus inbox/upstream **rdma-core** — version and firmware compatibility across driver, firmware, NCCL, and CUDA is a real operational matrix; “works on this kernel/firmware combo” is checked, not assumed.
- **nvidia-peermem** or kernel DMA-BUF for GPUDirect RDMA.

7.2 The toolbox — commands and what each tells you

Tool	What you use it for
ibstat / ibstatus	Adapter state, port state (Active/Down), rate, LID (IB), link layer (IB vs Ethernet)
ibv_devinfo	Verbs-level device view: ports, MTU, GUIDs — confirms RDMA is actually usable, not just link-up
ibdiagnet	IB fabric-wide diagnostic sweep: topology, errors, speed mismatches (IB clusters)
mlxconfig	NIC firmware configuration: port mode (IB/ETH), SR-IOV enable and VF count, RoCE options, PCIe settings — changes persist across reboots and differ from runtime config
mlxlink	The layer-1 microscope on the NIC side: link state, speed, FEC mode, per-lane FEC/BER counters, eye margins, module DOM — your first stop for suspected marginal cable/optic
ethtool -S / -i / --show-fec	Driver/firmware versions, ring settings, and the crucial per-NIC counter set (below)
devlink (health, params, port)	Modern kernel interface for NIC health reporters, firmware params, eswitch mode
rdma (from iproute2) / rdma stat	RDMA device/link state and protocol counters without vendor tools
perftest suite (ib_write_bw , ib_read_bw , ib_send_lat ...)	Raw RDMA bandwidth/latency between two hosts — isolates fabric+NIC from NCCL and the application; your baseline and bisection tool
nccl-tests (all_reduce_perf ...)	End-to-end collective performance as the workload sees it — the acceptance test
nvidia-smi topo -m	GPU/NIC PCIe affinity matrix — the NUMA/GPUDirect placement truth table
lspci -tv , lstopo (hwloc)	PCIe tree and NUMA layout; verify link width/speed (LnkSta) — a Gen5 x16 NIC trained at x8 is a silent half-bandwidth straggler
numactl , taskset	Pin benchmarks and daemons to the NUMA node local to the NIC under test

Counters worth knowing by name (from **ethtool -S** on **mlx5**): **rx_prio3_pause** / **tx_prio3_pause** (PFC activity per priority), **np_cnp_sent** / **rp_cnp_handled** (DCQCN loop health from receive and transmit side),

`out_of_sequence`, `packet_seq_err`, `local_ack_timeout_err` (loss/retransmission symptoms on RC QPs), `rx_discards_phy` (host not draining fast enough). These NIC counters plus switch counters bracket any problem between “fabric” and “host.”

7.3 NUMA locality and PCIe topology

A dual-socket GPU server is really **two computers sharing sheet metal**. Every high-speed device hangs off a specific socket’s PCIe root complex; crossing sockets means traversing the inter-CPU link (UPI/xGMI) — added latency, contention, and bandwidth ceilings.

Rules of the road:

- **GPU and its NIC must share PCIe locality** — same root complex, ideally the same PCIe switch. `nvidia-smi topo -m` legend: PIX (same switch) and PXB (same root complex, through switches) are good for GPUDirect; PHB, NODE, and SYS mean the path crosses the root complex or sockets — expect degraded peer-to-peer.
- **PCIe bandwidth budget:** Gen5 x16 is 63 GB/s raw — comfortably feeding a 400G NIC (50 GB/s); at 800G (ConnectX-8 class), PCIe Gen6 or wider host interfaces come into play. A slot mis-trained to x8, a riser problem, or a Gen4 negotiation silently halves this — check `LnkSta` versus `LnkCap` in `lspci -vv`.
- **ACS/IOMMU pitfalls:** PCIe Access Control Services on switches force peer-to-peer TLPs up through the root complex for isolation — killing GPUDirect P2P bandwidth. Platform tuning (disabling ACS where the trust model allows, correct IOMMU settings) is a standard build step; virtualized/multi-tenant designs need ATS-aware alternatives.
- **IRQ affinity:** steer NIC interrupt vectors to cores on the NIC-local NUMA node (`mlnx_tune / set_irq_affinity.sh` or manual `/proc/irq` maps). Completion processing on far-socket cores adds tail latency to every operation.
- **Housekeeping:** hugepages for pinned buffers/DPDK-style workloads, disabling deep C-states/frequency scaling on latency-critical cores, and keeping data-loading traffic (front-end NIC) off the rails.

7.4 SR-IOV and multi-tenancy

SR-IOV carves one physical NIC into virtual functions (VFs) with hardware-level isolation — the basis for giving containers/VMs direct RDMA access without a hypervisor in the data path. Enabled in firmware via `mlxconfig` (`SRIOV_EN`, `NUM_OF_VFS`), consumed in Kubernetes through the SR-IOV device plugin (Chapter 9). BlueField DPUs extend this model: the DPU owns the network stack (OVN/OVS offload, storage emulation, isolation) and presents clean interfaces to an untrusted host — increasingly common in multi-tenant AI clouds.

7.5 A host-side triage playbook

A concrete sequence you can narrate in an interview when “one server is slow”:

1. `nvidia-smi topo -m` — are GPU-NIC pairings PIX/PXB as designed?
2. `lspci -vv` (`LnkSta`) — every NIC and GPU trained at full width/generation?
3. `ethtool -i / ofed_info -s` / firmware — do driver/firmware/NCCL versions match the qualified matrix?
4. `mlxlink -d <dev> -m -c` — per-lane FEC and DOM on the suspect port: is layer 1 clean?
5. `ethtool -S` — pause counters exploding? `out_of_sequence` /retransmit symptoms? CNPs flowing?
6. `ib_write_bw` NIC-to-NIC pinned with `numactl` — raw RDMA at line rate? If yes, fabric+NIC are exonerated; look up-stack (NCCL config, CPU, dataloader).
7. `all_reduce_perf` at increasing scope (intra-node → same-rail pair → across pod) — find the boundary where bus bandwidth collapses; the boundary names the culprit.

INTERVIEW SOUNDBITE

“My demarc doesn’t end at the switchport — it ends at GPU memory. When a job is slow I bisect with `perfctest` before anything else: if `ib_write_bw` between the two hosts hits line rate, the fabric and NICs are innocent and it’s NCCL, NUMA, or the application; if it doesn’t, I have a two-node reproducer I can chase with `mlxlink`, FEC counters, and switch telemetry. And I always check the boring things first — PCIe width,

GPU-NIC affinity, driver/firmware matrix — because half of ‘network problems’ in GPU clusters are a NIC trained at x8 or a pod scheduled across sockets.”

8 AI Fabric Observability and Failure Analysis

Enterprise monitoring answers “is the network up and how busy is it?” AI fabric observability answers a harder question: “**is any job network-stalled, and which component is responsible?**” The defining incident type is not an outage — it is “this training job is 17% slower than yesterday,” with every port green.

8.1 The counter set that actually matters

Signal	Why it matters / what it tells you
PFC pause frames (tx/rx, per port, per priority)	The safety valve firing. Sustained pause TX from a switch = its buffers are stressed; pause RX = your neighbor is stressed. Rising trends localize congestion; a single host emitting constant pause = broken NIC endangering the pod.
ECN marked-packet rate (per queue)	The control loop working. Some marking under load is healthy; mark rate trends are your earliest congestion signal and your ECN-threshold tuning feedback.
Queue depth watermarks / buffer occupancy	Peak (not average) queue depth per port per class — the direct measurement of microbursts. Watermark polling or streaming beats any utilization graph.
Drops by reason (WRED, tail, ingress admission)	On a lossless class, any drop is an event worth explaining — usually QoS misclassification (Chapter 3) or headroom exhaustion.
FEC corrected/uncorrectable, per lane; DOM trends	Physical-layer pre-failure signal (Chapter 6). The “which optic hurts jobs next week” feed.
Link flaps, CRC, symbol errors	Job-restart-level events; correlate across shared panels/conduits/power.
NIC RoCE counters (<code>out_of_sequence</code> , <code>packet_seq_err</code> , timeouts, CNPs sent/handled, retransmits)	The end-system’s verdict on the fabric: retransmit symptoms prove loss/reordering that switch counters may not attribute; CNP volume shows the DCQCN loop’s workload.
Per-job/per-tenant network utilization	Attribution: which job drives which links; capacity and chargeback truth.
NCCL timing and GPU idle time (DCGM)	The business metric. Collective completion times and “GPU waiting on communication” percentage convert network counters into dollars.

8.2 Telemetry architecture

- **gNMI streaming telemetry** replaces SNMP polling: subscriptions push counters at 1–10 s cadence (watermarks even faster) into a pipeline (collector → time-series DB → dashboards/alerting — Prometheus/Grafana-style or vendor equivalents: Arista CloudVision/TerminAttr, Cisco MDT, SONiC’s gNMI container). Your Splunk/Zabbix experience maps directly — the delta is cadence, cardinality, and **queue-level** rather than interface-level data.
- **On-ASIC burst tooling**: watermark histograms, mirror-on-drop / What-Just-Happened-style event streams (packet + drop reason + queue snapshot), sampled INT/postcards. This class of tooling is how you **see** a 50-microsecond incast that a 30-second counter never will.
- **Host-side pipeline**: node exporters for `ethtool` / `rdma stat` counters plus DCGM for GPU metrics, sharing a time base with fabric telemetry so counters and job phases align.
- **Synthetic probes**: scheduled `perftest` / `nccl-tests` canaries across representative paths and rails — the fabric’s continuous “selftest,” and the pre/post gate for changes (Chapter 9).

KEY CONCEPT

The unifying discipline is **correlation by time and topology**: every alert, counter, job event, and cable belongs to a shared topology model and a shared timeline. “Rank 213 slow at 14:32” must join against “leaf-7 port 18 FEC burst at 14:31” in seconds, not via a human swivel-chairing between five dashboards. Building that join — SoT-backed topology plus time-aligned fabric/host/job telemetry — is one of the highest-leverage things a senior engineer delivers.

8.3 Worked example: “Why is this job 17% slower?”

The methodology, as you would narrate it in an interview:

1. **Scope from the job inward.** Get job facts first: which ranks/hosts, which collective is slow (NCCL logs, framework profiler), when it started, gradual or step-change. NCCL debug or profiler output showing all-reduce time inflated on specific ranks turns “the network is slow” into a bounded search.
2. **Straggler or systemic?** Compare per-rank timings. One or a few slow ranks → host/link/rail hunt (go to 3). Uniformly slow → fabric-wide or config-change hunt (go to 5).
3. **Straggler path:** for the slow rank’s host — `nvidia-smi topo`, PCIe `LnkSta`, thermals; NIC counters (`out_of_sequence`, pause, CNPs); `mlxlink` FEC per lane; the switchport’s FEC/DOM/queue counters. Classic finding: FEC uncorrectable bursts on one rail link — every collective crossing it stalls, and the whole job inherits that tail.
4. **Reproduce below the application:** `ib_write_bw` / `all_reduce_perf` over the suspect path. If synthetic reproduces it, bisect by moving one endpoint until the bad hop/link is isolated. If it doesn’t reproduce, it’s workload-level (dataloader, checkpoint interference, noisy neighbor).
5. **Systemic path:** change correlation first — config pushes, firmware, NCCL/driver versions, **and job placement** (did the scheduler split this job across pods today?). Then fabric health deltas versus last week: ECN mark rates, pause totals, watermark peaks, ECMP imbalance (per-member link utilization spread), new elephant tenants sharing the class.
6. **Close the loop.** Whatever was found becomes a permanent detector: a counter alarm, a CI check, a placement rule. The fleet learns; the 17% never recurs silently.

8.4 Failure analysis culture

- Blameless, evidence-driven postmortems, with **counter forensics** attached (the flap’s FEC history, the pause trend before deadlock, the mark-rate change after tuning).
- Fleet thinking: one weird optic is a component; the same optic part number failing at 3x baseline across the fleet is a vendor engagement and a proactive replacement campaign.
- Cost fluency: translate findings into GPU-hours. “That marginal link cost roughly 400 idle GPU-hours a day” is the sentence that gets physical-layer budget approved.

INTERVIEW SOUNDBITE

“My monitoring philosophy changed from ‘is the port up and how busy is it’ to ‘prove the job isn’t network-stalled.’ Concretely that means streaming queue watermarks, ECN mark rates, PFC pauses, and per-lane FEC into the same timeline as NCCL and GPU idle metrics, with a topology model to join them. Utilization averages are almost useless here — the events that cost money live at microsecond timescales and in the tails, so I instrument for bursts and stragglers, and I keep synthetic RDMA canaries running so the fabric tells me it degraded before a training team does.”

9 Orchestration, NOS Diversity, and Fabric-as-Code

9.1 Kubernetes: how the fabric reaches the pod

Most AI platforms schedule training via Kubernetes (or Slurm, often Slurm-on-K8s). The network engineer's concern: **how do RDMA NICs and GPUDirect capabilities get exposed to containers correctly?**

- **NVIDIA Network Operator** — the keystone component. It deploys and lifecycle-manages, as Kubernetes resources: the DOCA/OFED driver (as a container, decoupled from host image), the **RDMA shared device plugin**, the **SR-IOV device plugin**, and secondary network attachment for RDMA traffic. It pairs with the **GPU Operator** (drivers, DCGM, `nvidia-peermem`) so GPUDirect RDMA works end to end.
- **Multus** — pods get their default cluster network (the primary CNI) **plus** secondary high-performance interfaces via `NetworkAttachmentDefinition`: SR-IOV VFs, macvlan/ipvlan on the rail interfaces, or host-device passthrough. The training pod does NCCL over the secondary rails, not the cluster CNI.
- **Resource semantics**: NICs/VFs are schedulable resources (`nvidia.com/hostdev`-style requests via device plugins). Topology-aware scheduling and NUMA alignment (Topology Manager) keep the allocated GPU, NIC, and CPU cores on the same root complex — Chapter 7's rules, enforced by the scheduler.
- What you own in interviews: the flow “pod requests GPU + RDMA resource → device plugin allocates VF/device on the right rail → Multus attaches it → NCCL discovers it (`NCCL_IB_HCA`, `NCCL_SOCKET_IFNAME`) → traffic rides your QoS-engineered class.”

9.2 Slurm: the HPC scheduler view

Slurm remains standard in HPC-style shops: `gres` declares GPUs/NICs per node; `topology.conf` describes the switch hierarchy so the scheduler packs jobs into the smallest enclosing switch domain (exactly the failure-domain placement from Chapter 5); prologs/epilogs run health checks (including NIC/link state) before and after each job. The network team feeds Slurm its topology truth and consumes its job-placement data for observability correlation.

9.3 NOS diversity: SONiC, whitebox, and life beyond appliance CLI

AI shops mix NOS ecosystems in ways enterprise rarely does:

- **SONiC** — open-source NOS (Microsoft-originated, now broadly adopted; hyperscalers and increasingly enterprise AI). Architecture you should be able to sketch: containerized daemons (BGP/FRR, `orchagent`, platform containers) around a central Redis `CONFIG_DB/APPL_DB/STATE_DB`, with `syncd` programming the ASIC through **SAI (Switch Abstraction Interface)** — the vendor-neutral ASIC API. Config is JSON in Redis, not a config file you “write mem” on; operations are Linux-native (it is Debian), and telemetry/config surface is gNMI/gNOI.
- **SAI limitations** — the abstraction covers the common denominator; ASIC-specific buffer/QoS knobs may lag or need vendor extensions. When an AI-fabric tuning guide says “set this Tomahawk register behavior,” you need to know whether your NOS exposes it.
- **FBOSS / custom NOS** — Meta-style fully in-house stacks; relevant to hyperscaler roles primarily as context.
- **Traditional NOS in AI mode** — Arista EOS (deep buffer visibility, LANZ, AI-tuned profiles), Cisco Nexus/NX-OS and Silicon One, Juniper QFX/Apstra: still very common, especially in “enterprise-to-AI” environments. The senior skill is **not** memorizing one CLI — it is understanding the ASIC behavior underneath and driving any NOS through structured APIs.
- **gNMI/gNOI** — the management plane: gNMI for streaming telemetry and declarative config (`Set / Subscribe` on YANG paths), gNOI for operations (reboot, cert rotation, ping, factory reset). In pipeline-driven fleets, humans read dashboards and merge PRs; only break-glass touches a CLI.

9.4 Fabric-as-code: the operating model

At 1,000–100,000 GPUs, manual CLI operation is not merely inefficient — it is the leading cause of the exact one-off inconsistencies (a missing QoS map, a wrong MTU) that create silent stragglers. The target operating model:

1. **Source of truth (SoT)** — NetBox/Nautobot or equivalent holds the intended world: devices, ports, cables (the rail map!), IPs, ASNs, QoS profiles. The SoT is authoritative; the network is a **rendering** of it.
2. **Golden config generation** — templates render per-device config from SoT data. No hand-edited device configs; changes are data changes.

3. **GitOps pipeline** — every change is a PR: peer review, then CI validates (lint, schema, config diff, topology simulation — containerlab/vendor digital twin, route/policy checks), then staged rollout with health gates and **automated rollback** on regression.
4. **ZTP** — new or RMA'd switches boot, fetch identity and golden config, and join the fabric without a console session; at AI-buildout scale (hundreds of switches per pod), this is the only way to hit schedules.
5. **Automated cabling validation** — LLDP neighbor data continuously diffed against the SoT cable plan. This is the mechanized answer to “one mis-cabled rail”: detected in minutes at build time, not discovered weeks later as a performance mystery.
6. **Performance CI (“pre/post” discipline)** — before and after every meaningful change: synthetic `ib_write_bw`/`ib_read_bw` path tests and `nccl-tests` baselines on a reference set of node groups, compared against stored baselines with regression thresholds. A change that passes config validation but drops all-reduce bus bandwidth 8% **fails** and rolls back.
7. **Continuous drift detection** — running config and operational state (FEC modes, PFC state, buffer profiles) reconciled against intent on a schedule.

FIELD GOTCHA

The pre/post NCCL baseline is the discipline that most distinguishes AI fabric operations from enterprise change management. Enterprise verifies **reachability** after a change; AI fabric teams verify **performance**. Many costly regressions — an ECN threshold typo, a buffer profile swap, a link running at half FEC health after re-seat — are invisible to ping and BGP state, and perfectly visible to a 60-second `all_reduce_perf` run.

INTERVIEW SOUNDBITE

“I treat the fabric as a rendering of a source of truth: NetBox holds the rail map and QoS intent, templates render golden configs, every change is a reviewed PR that has to pass CI — including synthetic RDMA and NCCL baselines pre- and post-change, with automated rollback. And I’m deliberately NOS-agnostic: whether it’s EOS, NX-OS, or SONiC programming the ASIC through SAI, the pipeline speaks gNMI and the source of truth doesn’t care whose logo is on the faceplate.”

10 Facilities, DCI, and Where the Standards Are Going

10.1 Power, cooling, and rack reality

AI network design is physically constrained in ways enterprise design never was. The network engineer who ignores facilities gets designs that cannot be built:

- **Rack power density:** enterprise racks run 5–15 kW; air-cooled GPU racks 30–50 kW; liquid-cooled rack-scale systems (GB200/GB300 NVL72 class) roughly 120–140 kW **per rack**. Power, not space, is the scarce resource — “just add another switch/rack” is frequently a **megawatt** question, not a U-space question.
- **Liquid cooling changes layout:** direct-to-chip liquid loops, manifolds, and CDUs occupy space and dictate rack rows; network gear may remain air-cooled inside liquid-cooled rows, creating airflow islands that must be engineered. Rear-door heat exchangers, containment, and airflow direction (port-side intake vs exhaust switches — order the right airflow SKU!) are design inputs.
- **Optics thermals:** Chapter 6’s kilowatt-per-switch of optics needs front-panel airflow; dense OSFP cages at the top of a hot rack run modules near thermal limits, raising BER. Thermal design is a **link reliability** input.
- **Cable physics at scale:** an 8-rail 1,024-GPU pod is thousands of cables. Copper DAC/AEC is thick and short-reach — it constrains switch placement (middle-of-rack/row designs to keep runs under limits); fiber needs bend-radius discipline, tray capacity planning, and labeled, SoT-matched runs. Cable weight and tray fill are genuine structural questions in dense builds.
- **Serviceability:** clearance to swap a 2 kg, hot, finned OSFP module or re-seat an MPO without disturbing neighbors; blast-radius-aware positioning of leaf switches; spare optics stocked **on site** because a missing spare is idle GPUs.

10.2 DCI for AI: connecting clusters and regions

Multi-site AI infrastructure (DR for checkpoints, data gravity, multi-region training experiments, or simply “the next building”) brings service-provider skills back to the fore — Cisco 8000, Juniper PTX, Arista R-series, Nokia-class platforms:

- **Coherent pluggable optics:** 400ZR/ZR+ (and 800ZR emerging) put coherent DWDM into router/switch ports — **IP-over-DWDM / routed optical networking** collapses the traditional transport layer for metro/regional reaches. Know the reach classes (ZR 120 km amplified point-to-point; ZR+ higher power/longer reach over line systems).
- **Deep buffers at the boundary:** checkpoint replication and dataset transfer are massive elephant flows hitting bandwidth steps (from multi-Tb/s fabric onto $n \times 400\text{G}$ WAN). Deep-buffer routing silicon (Jericho-class VOQ platforms) at the DCI edge absorbs these transitions; this is one of the few places deep buffers are the right answer.
- **MACsec / encryption at line rate:** model weights and training data are crown-jewel IP; inter-site links run MACsec (or IPsec at lower rates) at 400G line rate — a hardware capability you must specify at purchase, not bolt on.
- **Traffic classes on the WAN:** checkpoint replication (bulk, deadline-driven), dataset ingest, storage replication, and control traffic get explicit bandwidth contracts; a checkpoint push must not starve interactive/control paths — classic SP traffic engineering (SR-TE, RSVP-TE heritage) applied to AI flows.
- **Failure isolation:** sites must fail independently: fiber-diverse paths, no shared line systems for redundant links, and — critically — **training jobs designed so a WAN event costs a checkpoint interval, not a cluster**. Cross-site **synchronous** training is generally impractical (speed of light: 1 ms per 100 km one way; collectives at that RTT stall GPUs), so inter-site traffic is asynchronous: checkpoints, data, and hierarchical/async training schemes.

10.3 Standards direction: Ultra Ethernet and the road ahead

Track the **Ultra Ethernet Consortium (UEC)** — the industry effort (AMD, Arista, Broadcom, Cisco, Intel, Meta, Microsoft, HPE, and many more) to make Ethernet the definitive AI/HPC fabric. **UEC Specification 1.0 (2025)** spans NICs, switches, optics, and cables. The technical headlines a candidate should know:

- **UET (Ultra Ethernet Transport):** a modern RDMA transport replacing RoCE’s IB-heritage semantics — designed for **packet spraying** (out-of-order delivery is normal; NIC places data directly), **selective retransmission** (no go-

back-N), and fast **ephemeral connection setup** (no heavyweight QP handshake per peer) for jobs with massive fan-out.

- **Congestion management:** sender-based and receiver-credit-based schemes designed to run **without requiring PFC-lossless fabrics** — loss-tolerant by design, plus richer congestion signaling (beyond one ECN bit) and programmable congestion control.
- **Link-layer options:** LLR (link-level retry) and credit-based flow control on Ethernet links, in-network collectives (INC) — notice these are Ethernet adopting InfiniBand's best ideas while keeping the Ethernet ecosystem.
- **Why it matters to you:** the PFC/ECN/DCQCN engineering in Chapter 3 is the **current** generation. The direction of travel is fabrics that spray packets, tolerate loss, and self-manage congestion — vendor implementations (NVIDIA Spectrum-X adaptive routing + DDP, Broadcom's AI fabric features) are already partway there. A senior hire is expected to run today's fabrics **and** have a view on this transition.

INTERVIEW SOUNDBITE

"For multi-site AI my starting position is: synchronous training doesn't cross the WAN — physics says so — therefore DCI is engineered for asynchronous elephants: checkpoint and dataset replication over 400ZR+ routed-optical links, deep-buffer VOQ platforms at the bandwidth step-down, MACsec everywhere because model weights are the company's crown jewels, and traffic contracts so a checkpoint push can't starve control planes. And I'm tracking UEC closely — packet spraying with selective retransmit essentially retires the go-back-N problem that makes today's lossless engineering so delicate."

11 Interview Preparation: Questions, Model Answers, and Positioning

This chapter converts the technical material into interview performance. The model answers are deliberately senior in tone: they state the concept, the failure modes, and the operational discipline — the three layers interviewers listen for.

11.1 How to position an enterprise background

Lead with the mental-model shift (Chapter 1), then anchor every answer in your transferable strengths: routing at scale (the RoCE underlay is BGP), QoS depth (PFC/ECN is QoS with higher stakes), telemetry (gNMI is your Splunk instinct at higher cadence), and automation (fabric- as-code is IaC discipline applied to switches). Never apologize for lacking GPU-cluster mileage; demonstrate you know precisely **which** skills are new and show evidence you are building them (the lab work in Chapter 12 is that evidence).

11.2 Technical questions and model answers

Q: Why can't you just run RoCE on a default-configured enterprise switch?

A: RoCEv2 carries InfiniBand's reliable-connection transport, which traditionally recovers loss with go-back-N — one drop retransmits the whole window, stalling a collective that thousands of GPUs are waiting on. Default Ethernet drops on congestion, so I must engineer the fabric: consistent DSCP-to-traffic-class mapping end to end, ECN/WRED thresholds so DCQCN rate control acts first, PFC on the RoCE class as a safety net with properly sized headroom and a deadlock watchdog, and CNPs in a protected strict-priority queue. Then I monitor marks, pauses, and watermarks to prove it stays engineered.

Q: Explain DCQCN end to end.

A: Three roles. Congestion point: the switch marks packets CE via ECN as queue depth crosses thresholds. Notification point: the receiving NIC sees CE-marked RoCE packets and returns CNPs to the sender. Reaction point: the sending NIC cuts that queue pair's rate sharply on CNP arrival and recovers additively, governed by a decaying congestion estimate. Tuning is about the ECN thresholds — mark too early and you waste bandwidth, too late and queues grow until PFC fires. The design intent is ECN resolves congestion at RTT timescales so PFC almost never engages.

Q: What is a PFC storm and how do you contain it?

A: PFC pause propagates: a paused device's buffers fill, so it pauses its own upstreams, and congestion cascades backward — potentially from a single broken receiver that stopped draining. One malfunctioning NIC can degrade a whole pod. Containment: PFC watchdog to detect queues paused beyond a threshold and force-drain them, valley-free routing to avoid cyclic buffer dependency deadlocks, alarming on pause-frame rates so a chronically pausing endpoint gets quarantined, and keeping PFC scoped to only the RoCE class so the blast radius never includes control traffic.

Q: Why are AI fabrics rail-optimized?

A: Each of the eight GPUs in a server has its own NIC; rail k of every server connects to the same leaf or plane. NCCL is topology-aware and organizes inter-node traffic so GPU k talks predominantly to GPU k on other nodes — same rail, one leaf hop, no spine crossing — while cross-rail movement rides NVLink inside the chassis. It preserves

GPU-to-GPU bandwidth and cuts contention. The operational flip side: the cable map is a correctness constraint, so I validate cabling automatically against the source of truth — one mis-cabled rail silently degrades every job that touches it.

Q: A training job is 17% slower than last week. Walk me through your approach.

A: First scope from the job inward: which ranks, which collective, when it started — NCCL logs and per-rank timings tell me if it's a straggler or systemic. Straggler: that host's PCIe width and GPU-NIC affinity, NIC counters for retransmit symptoms and pauses, per-lane FEC and DOM on its links — the classic culprit is a marginal optic on one rail. Systemic: correlate changes — config pushes, firmware, NCCL versions, and whether the scheduler placed the job across pods — then compare fabric baselines: ECN mark rates, pause trends, watermark peaks, ECMP member imbalance. I reproduce below the application with `ib_write_bw` and `all_reduce_perf` to bisect fabric versus host versus workload, and whatever I find becomes a permanent alarm or CI check.

Q: Why does static ECMP struggle with AI traffic, and what are the fixes?

A: Collectives generate a handful of enormous, long-lived, synchronized flows — almost no entropy for a five-tuple hash, so two elephants land on one uplink while neighbors idle: hot link, marks, pauses, stragglers, at 40% average utilization. Fixes in escalation: more entropy (per-QP UDP source ports, multiple QPs per connection), flowlet-based dynamic load balancing that re-paths in the natural gaps, and ultimately packet spraying with NIC-side reordering — which is what IB adaptive routing and Spectrum-X do today and what Ultra Ethernet standardizes.

Q: How do you size PFC headroom, and what happens if you get it wrong?

A: Headroom absorbs in-flight data after XOFF: propagation delay both ways — so cable length matters — plus peer response time and the frames already serializing. Too little headroom means drops on a nominally lossless class, which is the worst failure mode because everyone assumes lossless means lossless; too much wastes shared buffer that other ports need. I use the platform's per-cable-length guidance, keep cable lengths in the source of truth, and alarm on any ingress drop in the lossless class as a design violation.

Q: What does GPUDirect RDMA require from the infrastructure?

A: An RDMA NIC and GPU with peer-to-peer PCIe visibility — same root complex, ideally the same PCIe switch, verified with `nvidia-smi topo`; ACS configured so peer TLPs aren't forced through the root complex; the peer-memory kernel module or DMA-BUF path loaded; and a qualified driver/firmware/CUDA/NCCL version matrix. Symptom of getting it wrong: bandwidth tests that hit line rate host-to-host but collapse GPU-to-GPU, because data is silently bouncing through system memory.

Q: InfiniBand or Ethernet for the next cluster?

A: Both are proven at frontier scale, so it's an engineering-economics decision. InfiniBand: losslessness by credit-based flow control with no tuning, mature adaptive routing, SHARP in-network reduction, one vendor's turnkey performance — at cost of a parallel skill set, single-vendor supply, and a separate operational stack. Ethernet/RoCE: ecosystem economics, multi-vendor leverage, one operational model with the rest of the plant, and with modern platforms — adaptive routing, NIC reordering, tuned DCQCN — comparable delivered performance; UEC is closing the remaining transport gaps. My answer for most organizations is Ethernet for strategic alignment

unless the workload profile and vendor relationship argue for IB — and I'd insist the decision include operations, not just benchmarks.

Q: What monitoring would you build for a new GPU fabric?

A: Streaming gNMI at seconds cadence into a time-series pipeline: per-queue watermarks, ECN marks, PFC pauses per priority, drops by reason, per-lane FEC corrected and uncorrectable, DOM trends. Host side: NIC RoCE counters — out-of-sequence, retransmit timeouts, CNPs — plus DCGM for GPU idle time, on the same timeline. Topology-joined, so a rank maps to a rail and a port in one query. Then synthetic canaries — scheduled perf test and small NCCL runs — as continuous proof of fabric performance, and as the pre/post gate in the change pipeline. The design goal is answering 'is any job network-stalled' in minutes.

Q: How does your change management differ on an AI fabric?

A: Everything is a PR against a source of truth, rendered to golden configs, validated in CI — but the AI-specific discipline is performance regression testing: before and after every change we run RDMA and NCCL baselines on reference paths and compare against stored results with thresholds. Reachability tests don't catch an ECN threshold typo or a buffer profile regression; a sixty-second all-reduce benchmark does. Plus automated rollback, canary scope-outs, and drift detection reconciling operational state — FEC modes, PFC state — against intent.

Q: What is SHARP and why does it matter?

A: In-network computing: InfiniBand switches perform the reduction arithmetic of all-reduce inside the fabric, aggregating as data flows up a tree and multicasting results down. It roughly halves the data movement and removes latency stages, which matters most at large scale and for frequent small collectives. It's also a preview of where Ethernet is headed — UEC defines in-network collectives, and switch-assisted reduction is a differentiator vendors compete on.

Q: What would you check before handing a new fabric to the ML platform team?

A: A formal acceptance suite: cabling validated by LLDP diff against the rail map; every link at full speed and width with clean per-lane FEC baselines and DOM within thresholds; QoS rendering verified on every device — classification, ECN thresholds, PFC state, headroom; underlay routing convergence tested with failure injection; then performance acceptance — `ib_write_bw` full-mesh sweeps per rail, then `nccl-tests` all-reduce and all-to-all at increasing scope from single rack to full pod, compared against the design's expected bus bandwidth. All results stored as the fleet's birth certificate for future regression comparison.

Q: Where do you see AI network engineering going in the next few years?

A: Transport: Ultra Ethernet — packet spraying, selective retransmission, congestion control that doesn't require fully lossless fabrics — begins retiring today's delicate PFC-centric engineering. Physical: 1.6T ports, linear-drive and eventually co-packaged optics to contain power. Scale-up and scale-out blur as NVLink-class domains grow and fabrics carry more of the collective load in-network. And operationally, the job keeps moving toward fleet software engineering: the differentiator is the telemetry, validation, and automation stack, not CLI fluency.

11.3 Questions to ask the interviewer

Asking sharp questions signals seniority as strongly as answering them:

- “Ethernet or InfiniBand back-end, and what drove that choice? Are you tracking UEC for the next generation?”
- “How is the fabric validated after changes — do you run NCCL/RDMA performance baselines pre/post, or reachability only?”
- “What’s the current top network-attributed cause of job slowdown or restart, and how is it detected today?”
- “Is cabling/rail mapping validated automatically against a source of truth? What’s the SoT?”
- “Where does the network team’s ownership end — switchport, NIC firmware, host stack, NCCL tuning?”
- “What does the telemetry pipeline look like — gNMI cadence, queue watermarks, correlation with job metrics?”
- “How are jobs placed relative to failure domains, and does the network team feed the scheduler’s topology?”

11.4 Red flags to avoid in your own answers

- Talking about link utilization averages as evidence of health (signals enterprise reflexes).
- Claiming PFC alone makes a fabric lossless and healthy (ignores ECN’s primary role and PFC’s pathologies).
- Treating the host as out of scope (“that’s the server team”) — in this role the NIC and PCIe topology are yours.
- Vendor-war absolutism in the IB-versus-Ethernet question — the senior answer is engineering-economics, not tribal.
- Any suggestion that manual CLI operation scales to this environment.

12 A Practical Learning Path, Labs, Glossary, and References

12.1 The five-phase learning path

12.1.1 Phase 1 — Network transport (2–3 weeks)

Master Chapters 2 and 3 until you can whiteboard them: RoCEv2 encapsulation, RDMA verbs and queue pairs, InfiniBand’s credit-based losslessness, PFC mechanics and pathologies, ECN/DCQCN, DSCP-to-TC mapping, and congestion behavior. Concrete exercises:

- Read a vendor AI/RoCE deployment guide end to end (Arista/Broadcom AI networking guide, NVIDIA RoCE configuration guides) and reproduce its QoS design rationale in your own words.
- **No-hardware lab:** Soft-RoCE (`rdma_rxe`) turns any Linux VM NIC into an RDMA device: `rdma link add rxe0 type rxe netdev eth0`, then run `ib_write_bw`/`ib_send_lat` between two VMs and watch verbs, QPs, and completions with `rdma stat`. Semantics without the hardware.
- Explain to a colleague why go-back-N makes loss catastrophic for RC transport — if you can teach it, you own it.

12.1.2 Phase 2 — GPU workload awareness (2–3 weeks)

Master Chapter 4: collectives, NCCL, parallelism strategies, and traffic signatures.

- Run PyTorch distributed on CPUs (Gloo backend) or any small GPU setup: implement data-parallel training of a toy model, log where time goes per step.
- Build `nccl-tests` and study the output columns until algorithm bandwidth versus bus bandwidth is second nature; learn `NCCL_DEBUG=INFO` output — ring/tree construction, chosen interfaces, rails.
- Hand-derive ring all-reduce cost: N GPUs, S bytes each — reduce-scatter plus all-gather, each moving $S \times (N-1)/N$ per GPU. Interviewers ask this.

12.1.3 Phase 3 — Build/test muscle (3–4 weeks)

Get hands on RDMA hardware if at all possible: two to four Linux servers with ConnectX NICs (used ConnectX-5/6 cards are affordable; two direct-attached hosts suffice to start).

- Full `perftest` matrix: `ib_write_bw`, `ib_read_bw`, `ib_send_lat` across message sizes; pin with `numactl` and measure the NUMA penalty yourself.
- NIC forensics drill: `mlxconfig -d <dev> query`, `mlxlink` per-lane FEC/DOM readout, `ethtool -S` counter walk before and after a saturating run — find the pause and CNP counters.
- If a managed switch is available: configure the RoCE QoS class (DSCP 26 to queue 3, ECN thresholds, PFC on the class, CNP strict-priority), then **deliberately misconfigure one element** and observe the counter signature of each failure mode. This drill builds the diagnostic intuition Chapters 3 and 8 describe.
- `nccl-tests` over the fabric; compare bus bandwidth against line rate; degrade a link (FEC-stressed cable, forced lower speed) and watch the straggler effect.

12.1.4 Phase 4 — Fabric operations (3–4 weeks)

Practice operating like Chapter 9:

- Model a small fabric (even your lab) in NetBox: devices, interfaces, cables; write a script diffing LLDP neighbors against it — you have now built cabling validation.
- Stand up gNMI collection (or SONiC’s telemetry container, or `gnmic` against a vEOS/cEOS lab) into Prometheus/Grafana; dashboard queue watermarks, pause counters, ECN marks.
- Build a containerlab topology with SONiC or cEOS nodes; template configs from the NetBox data with Jinja2; put the render-validate-deploy loop in CI (GitHub Actions is fine).
- Write the pre/post harness: a script that runs `ib_write_bw` and `all_reduce_perf` across defined pairs, stores JSON results, and diffs against baseline with thresholds.

12.1.5 Phase 5 — Vendor depth (ongoing)

Pick one AI stack and one Ethernet stack for depth; stay conversational on the rest:

- **Strong combo:** NVIDIA Spectrum-X + InfiniBand ecosystem (Cumulus/NVOS, UFM, SHARP, ConnectX/Blue-Field) **plus** Arista EOS or SONiC/Broadcom. Cisco Nexus/Silicon One and Juniper QFX/PTX/Apstra matter especially in enterprise-to-AI shops.
- Certifications/curricula that map well: NVIDIA's AI networking certifications (associate and professional level), vendor AI fabric design guides, and the UEC 1.0 specification overview documents.
- Follow the primary sources: UEC working-group publications, NVIDIA networking documentation, Arista/Broadcom AI deployment guides, SONiC release notes, and large-scale training infrastructure papers from Meta, Microsoft, and Google — these papers are effectively free senior-level case studies.

12.2 Glossary

- **All-reduce** — Collective where every GPU obtains the sum of all GPUs' data
- **All-to-all** — Every GPU sends distinct data to every other GPU (MoE routing)
- **BER** — Bit error rate; pre-FEC and post-FEC rates assessed separately
- **Bisection bandwidth** — Bandwidth across a worst-case halving of the fabric
- **CNP** — Congestion Notification Packet — DCQCN's feedback message
- **Collective** — Group communication operation synchronizing GPUs
- **DCQCN** — ECN-driven congestion control for RoCEv2, in NIC hardware
- **DDP** — Direct data placement — NIC writes out-of-order packets to correct memory
- **DOM/DDM** — Digital optical monitoring — per-lane optic health telemetry
- **ECMP** — Equal-cost multipath; hash-based, struggles with elephant flows
- **ECN** — Explicit Congestion Notification — marking instead of dropping
- **FEC** — Forward error correction; RS-544 mandatory at PAM4 rates
- **Flowlet** — Burst of a flow separated by idle gaps; re-pathable safely
- **GPUDirect RDMA** — NIC DMAs directly to/from GPU memory, bypassing host RAM
- **Go-back-N** — Retransmission of the entire window after one loss
- **Headroom** — PFC buffer reserve absorbing in-flight data after pause
- **HOL blocking** — Head-of-line blocking — paused class stalls innocent flows
- **Incast** — Many senders converging on one receiver simultaneously
- **LID** — InfiniBand local identifier assigned by the subnet manager
- **LPO** — Linear pluggable optics — no module DSP, lower power
- **MFU** — Model FLOPS utilization — the cluster efficiency metric
- **MPO/MTP** — Multi-fiber connector for parallel optics; top contamination point
- **NCCL** — NVIDIA's topology-aware collective communication library
- **NVLink domain** — GPUs connected by NVLink/NVSwitch (server or NVL72 rack)
- **OSFP/QSFP-DD** — 800G-capable pluggable module form factors
- **PAM4** — 4-level signaling, 2 bits per symbol; needs FEC
- **PFC** — Priority Flow Control (802.1Qbb) — per-class pause
- **PFC watchdog** — Breaks deadlocks by draining chronically paused queues
- **PXN** — NCCL routing via NVLink to exit on the correct rail
- **QP** — Queue pair — RDMA's connection endpoint
- **Rail** — NIC k of every server, cabled to a common leaf/plane
- **RoCEv2** — RDMA over Converged Ethernet — IB transport over UDP/IP (port 4791)
- **SAI** — Switch Abstraction Interface — NOS-to-ASIC API (SONiC)
- **SHARP** — IB in-network reduction — switches compute the all-reduce
- **SM** — InfiniBand subnet manager (OpenSM/UFM)
- **SoT** — Source of truth — intended-state database (e.g., NetBox)
- **Straggler** — Slowest participant that stalls a synchronized collective
- **UEC/UET** — Ultra Ethernet Consortium / its new RDMA transport
- **Verbs** — The RDMA programming API (libibverbs)
- **WRED** — Weighted random early detection — the ECN marking mechanism

12.3 References

1. NVIDIA Spectrum-X Ethernet Platform for AI Networking — <https://www.nvidia.com/en-us/networking/spectrumx/>
2. Arista/Broadcom, High-Performance Ethernet Networking for Artificial Intelligence (AI deployment guide) — <https://www.arista.com/assets/data/pdf/Arista-Broadcom-AI-Networking-Deployment-Guide.pdf>
3. NVIDIA Collective Communications Library (NCCL) — <https://developer.nvidia.com/nccl>
4. NVIDIA Network Operator, Deployment on Kubernetes — <https://docs.nvidia.com/networking/display/kubernetes2570/deployment-guide-kubernetes.html>
5. Ultra Ethernet Consortium, Specification 1.0 announcement — <https://ultraethernet.org/ultra-ethernet-consortium-uec-launches-specification-1-0-transforming-ethernet-for-ai-and-hpc-at-scale/>

Further primary sources worth bookmarking: NVIDIA networking documentation (RoCE configuration, GPUDirect), `perftest` and `nccl-tests` GitHub repositories, SONiC project documentation, RFC 7938 (BGP in large-scale datacenters), and the DCQCN paper (“Congestion Control for Large-Scale RDMA Deployments,” SIGCOMM 2015).

12.4 Closing

The distance between a strong enterprise network engineer and a strong AI datacenter network engineer is real but bounded: one new transport model, one new congestion discipline, one new workload to internalize, one physical layer to respect, one host stack to own, and an operational culture that measures itself in GPU idle time. Every one of those is learnable with the path above — and the enterprise discipline you already have (rigor, change safety, telemetry instinct, automation) is precisely what AI infrastructure teams are desperate to hire.